Bevezetés a Rust programozásba

Gembela Gergely

8. óra: Konkurens és async programozás, Tokio

https://github.com/bmeaut/VIAUBXAV090

Konkurencia vagy párhuzamosság

- A konkurencia több feladat kezelése egyszerre
  - Megvalósulhat egy szálon (/fizikai processzormagon) is
- A párhuzamosság a feladatok konkrét egyidejű/párhuzamos futtatása több processzormagon/szálon

Async programozás

- Vannak műveletek, amelyek nem egy számítás eredményét adják vissza, hanem egy *future*-t
  - A future (más nyelvekből Task vagy Promise néven is ismerhetitek) egy olyan érték, amely segítségével előbb-utóbb a tényleges eredményhez tudunk jutni

Async Rust

- Régóta fejlesztett nyelvi elem(ek gyűjteménye)
  - Továbbra is fejlődik, nem végleges
  - Elég sok része elérhető a stabil fordítóban is
    - Sok helyen használják prodban is
  - Több megvalósítás (runtime): `Tokio`, `smol`, `async-std`, `embassy`
  - A kiválasztott runtime-ot, és jellemzően a `futures` crate-et is a mi feladatunk telepíteni

A runtime feladatai

- Reactor: OS interakció, események
- Ütemezés (mikor, és melyik szálon futhat egy task)
- Executor: a runtime felhasználó felé biztosított felülete
- Sok esetben teljesen más kiegészítőket is kapunk az adott runtime-tól

A `Future` trait

- Az `async` és `await` kulcsszavak hatására a fordító átalakítja a kódot a `Future` trait használatával
- Ha elég jól ismerjük a Rust async képességeit (pl. `Context`, `Waker` típusokat és a `poll` működését, sőt, esetenként a választott async runtime-ot), kézzel is implementálhatjuk
- Egy `Future` teljesülésére az `await` kulcsszóval várhatunk
- Máshogy érdemes a rendszert elképzelni, mint a .NET Task-jait

A `Future` trait

- Lényegében egy állapotgép, amely `Pending` amíg további munkát kell végeznie, majd `Ready` állapotba kerül
- "Lazy", tehát nem csinálnak semmit, amíg valaki el nem kezd rájuk várni
  - Lényeges különbség a szálakhoz képest!
  - így csak akkor végez munkát, ha szükség is van az eredményére
- Az `async` függvényeink számára anonim típus generálódik

Mi valójában egy `async fn`?

- Egy `async move` blokkba csomagolja a függvény testét
  - Ez egy kifejezés, ami ráadásul `impl Future`

```
async fn a1(p1: &str) -> String {/*...*/}
//átalakítva
fn a1(p1: &str) -> impl Future<Output = String> {
  async move {/*...*/}
}
```

Rust `await`

- Rustban az `await` kulcsszó használatával megvárhatjuk, hogy egy `Future` eredménye előálljon
  - A future többször ellenőrzésre kerül, hogy elkészült-e (ez a folyamat a polling)
- Postfix kulcsszó, tehát nem `await future`, hanem `future.await` a szintaxis

A `Tokio`

- „Egy runtime megbízható és gyors hálózati alkalmazások készítéséhez” (docs.rs)
  - Ennél mára jóval több
  - Nagy szerepe volt az async elemek fejlesztésében is
  - Async IO API-ok pl. TCP/UDP socketek, fájlműveletek, vagy folyamatközi kommunikáció kezelésére

`async` blokkok

- Maga is egy kifejezés (expression)
- A `return`, `break`, és `continue` kulcsszavak nem mutathatnak rajtuk kívül álló kontextusra

```
let b1 = async {/*...*/};
let b2 = async move {/*...*/};
```

async closure

- Egészen új feature (1.85+), pedig lényegében azonnal természetes részét képezte az async Rustnak
- Nem keverendő a closure-be csomagolt async blokkal

```
let c1 = async || {/*...*/};
let c2 = async move || {/*...*/};

let binc = || async{}; //Ez nem async closure, hanem async blokk closure-ben
```

Alternatív runtime-ok

- A nyelvtől független az async runtime, a Tokio-n kívül érdekes pl.
  - `smol`: egy egyszerűségre törekvő async runtime
  - *`async-std`*: discontinued, de rengeteg (1700+) könyvtár használja
  - `Embassy`: egy kifejezetten beágyazott környezetre fejlesztett keretrendszer, async runtime-ként is viselkedik
- További pl. OS/kernel specifikus runtime-ok (pl. `glommio`)

Streamek

- Egy aszinkron értékekből álló sorozat
  - A standard könyvtár `Iterator` trait async megfelelője
  - Adapterek segítségével sok mindent streammé alakíthatunk
- Az `async_stream` crate segítségével a .NET-ből ismerős yield-éhez hasonló funkcionalitást kaphatunk

```
while let Some(value) = s.next().await { //A nexthez itt szükség lesz a megfelelő
    println!("got {}", value); //trait extension importálására
}
```

`AsyncRead` és `AsyncWrite`

- Traitek, amelyek lehetővé teszik byte streamek async írását és olvasását
- Általában nem kézzel hívjuk rajtuk az olvasást/írást, de pl. hálózati kommunikáció megvalósításakor előfordul
- A `Vec<u8>`, és `&mut [u8]` is megvalósítják
- `io::split()` segítségével bármi író és olvasó részre bontható, ami `AsyncRead + AsyncWrite`

Egy projekt felkészítése `async` használatára

- A kiválasztott runtime-ot fel kell venni a függőségek közé
- Az `async` kód futtatásához
  - Vagy módosítanunk kell a main függvényt (pl. `#[tokio::main]` segítségével)
  - Vagy valahol létre kell hoznunk egy runtime-ot, és megkérnünk, hogy kezdje el az async blokkunk/függvényünk végrehajtását

Példa: TCP echo server

- A példa célja egy Tokio TCP socket páros használatával egy echo szerver megvalósítása
- A szerver tetszőleges porton futhat, és a válasza azonos kell hogy legyen a fogadott üzenettel
- A kliens ne blokkolja közvetlenül a fő szálat amíg a válaszra vár

Példa: rendkívül egyszerű HTTP server `axum`-mal

- Ebben a példában pár soros HTTP szervert készítünk
- A cél egy egyszerű 1-1 `POST` és `GET` műveletet támogató szerver készítése
- A példa segít a W8T1 megoldásában

W8T1: Táblakezelés: lekérdezés és beszúrás REST API felett

- A cél az eddigi hetekben létrehozott egyszerű táblakezelő megoldás számára REST API készítése
- Javaslom az `axum` használatát

Források, további anyagok

- The State of Async Rust: Runtimes
- wg-async
- Rust 2024 (async closures)
- Tokio tutorial
- Asynchronous Programming in Rust

Köszönöm a figyelmet!